# iMouse: Calibrated Facial Gestures and Adaptive Cursor Control with a Webcam

Akshat Singhania (230102)
Deep Aryan Singh (230346)
Harsh Pratap Singh (230447)
Rohit Vinod Atkurkar (230872)
Indian Institute of Technology Kanpur
EE604
https://github.com/harshpsi9194/EE604_Course-Project

## Abstract

*Hands-free desktop interaction with a standard webcam requires stable pointer movement and separation of intentional gestures from ordinary blinks. We extend an existing OpenCV–MediaPipe–PyAutoGUI controller with neutral calibration, normalized eyelid geometry, a temporal gesture state machine, gesture-aware pointer holds, and speed-adaptive filtering. The design retains the pretrained landmark detector and adds a latest-frame capture path and configurable removal of the mouse library's post-call pause. Its contribution is a task-specific integration and evaluation of established techniques, rather than a new landmark or gaze-estimation algorithm. Supplied offline validation reports a median read-complete-to-dispatch time of 5.33 ms on a repeated-image fixture. Reproduced synthetic tests generate no clicks during 100 bilateral blinks, one click for each of 60 deliberate winks, and reduce stationary coordinate standard deviation from 2.90 to 1.01 pixels. All 33 supplied regression tests pass in a fresh check. These results establish software behavior under controlled inputs; they do not establish live-user accuracy, clinical accessibility, or end-to-end webcam latency.*

## 1. Introduction

A physical mouse assumes that a user can continuously position and click a handheld device. A webcam interface offers an alternative input channel for hands-busy interaction and motivates exploratory accessibility work. However, tracking a facial landmark is only the first part of an interface. Natural blinks can resemble intentional clicks, small landmark fluctuations can shake the pointer, and serial processing or library sleeps can delay feedback. A useful controller must coordinate these competing actions and communicate its state.

The starting project maps an eye landmark to screen coordinates and uses facial gestures for mouse commands. We retain this lightweight architecture, with no additional model training, and address its control-layer weaknesses. We call the result a *head-assisted iris-position controller*: the pointer depends on iris position in the camera image, which also changes when the head moves. It does not estimate the point of gaze on a display. This distinction agrees with the limitations of MediaPipe Iris [2].

**Contribution and novelty.** The course-project contribution is the integration of person-specific neutral calibration, bilateral-blink cancellation, release-confirmed wink clicks, and cursor freezing into a single control policy, combined with adaptive smoothing and explicit latency instrumentation. EAR and One Euro filtering are prior methods [3, 4]. We claim an engineering extension of the supplied baseline, not global algorithmic novelty. The evidence supports deterministic gesture handling and synthetic stabilization; a matched human baseline comparison remains necessary to quantify practical benefit.

## 2. Related Work and Design Rationale

**Landmark-based perception.** MediaPipe Face Mesh provides facial landmarks, while refined eye landmarks enable iris-position features [1, 2]. We reuse its pretrained inference rather than collect a training dataset or train a new vision model. This lowers implementation effort but inherits the detector's sensitivity to visibility and image quality.

**Eye opening and blink detection.** Soukupova and Cech summarize eyelid geometry using the eye aspect ratio and use a temporal classifier for blink detection [3]. Our code

| Component | Supplied baseline | Revised controller |
|---|---|---|
| Pointer input | Iris boundary 477 | Mean centers 468, 473 |
| Click features | Fixed eyelid gaps | Calibrated EAR |
| Click decision | Per-eye triggers | Wink release; blink cancellation |
| Pointer stability | Direct mapping | Neutral gain + One Euro |
| Scroll | 300-unit bursts | 3-step rate-limited roll |
| Capture | Serial read | Latest-frame webcam slot |
| Mouth action | Exit | Sustained pause/resume |
| Measurement | Trigger counters | Stage timings + tests |

Table 1. Code-level comparison. These are implementation differences, not measured human performance gains.

uses the same geometric idea with calibrated thresholds and explicit temporal rules. It does not reproduce their SVM or claim their dataset results. The controller's task is to reject bilateral blinks and emit a desktop click after a deliberate unilateral wink.

**Filtering noisy input.** Casiez et al.'s One Euro filter changes its low-pass cutoff with input speed [4]. We use this established method to balance stationary jitter and movement responsiveness. A fixed moving average would also smooth noise but adds a fixed-window delay; adaptive filtering provides explicit controls for the speed/noise tradeoff.

**Desktop integration.** PyAutoGUI documents a default 0.1-second pause after calls and a corner fail-safe [5]. The revision makes this pause configurable, defaults it to zero, and retains the fail-safe. OpenCV handles image acquisition [6]. A bounded application-side frame slot prioritizes recent observations, although camera-driver buffering remains outside this guarantee.

## 3. Problem Definition

The input is a single RGB webcam stream containing one visible face. The output comprises primary-display cursor coordinates, left/right clicks, vertical scrolling, and a pause/resume state. The default request is $640 \times 480$ at 30 FPS; actual webcam properties depend on hardware and driver. The interface assumes adequate eye visibility, a stable camera, a roughly frontal pose, and voluntary control of the required gestures. Users who cannot wink independently may need an alternative click mechanism. No depth camera or training stage is required by this implementation.

## 4. Proposed Method

Figure 1 summarizes the controller. Perception supplies features; a state machine decides whether an action is permitted before pointer mapping and filtering.

### 4.1. Acquisition and Feature Geometry

A reader thread stores only the latest webcam frame, sequence number, and read-complete timestamp. The consumer can skip old frames rather than accumulate a processing backlog. File replay remains serial and uses video time for gesture durations. Each frame is resized only if necessary, mirrored, and converted from BGR to RGB. Face Mesh processes one face with refined landmarks and detection/tracking confidence thresholds of 0.6.

For each eye, use corners $p_1, p_4$ and eyelid pairs $(p_2, p_6)$ and $(p_3, p_5)$:

$$E = \frac{\|p_2 - p_6\|_2 + \|p_3 - p_5\|_2}{2\|p_1 - p_4\|_2}. \tag{1}$$

Distances use pixel coordinates $(xW, yH)$, avoiding distortion from treating normalized horizontal and vertical coordinates as equal units. The implementation uses indices $(33, 133; 160, 144; 158, 153)$ for one eye and $(362, 263; 385, 380; 387, 373)$ for the other. These left/right labels preserve the mirrored baseline mapping; `--swap-winks` swaps the buttons.

The iris feature is the mean of landmarks 468 and 473. Mouth opening is $M = \|p_{13} - p_{14}\|/\|p_{61} - p_{291}\|$. Head roll $\theta$ is the angle of the line joining eye-corner midpoints, computed with $\mathrm{atan2}$. Eye-corner midpoints reduce coupling between eyelid motion and roll. Invalid geometry or missing landmarks yields no features.

### 4.2. Neutral Calibration and Mapping

The application collects 45 consecutive valid samples with both EAR values above 0.12 and mouth ratio below 0.28. Invalid samples clear this collection. Componentwise medians define neutral iris coordinates $(x_0, y_0)$, open-eye values $(E_{L0}, E_{R0})$, and roll $\theta_0$. At 30 processed FPS, 45 samples take approximately 1.5 seconds; this is conditional on valid observations. The lower-level calibration helper permits at least 15 samples, while the application uses 45.

For screen size $(S_w, S_h)$ and gains $(g_x, g_y)$:

$$u = [0.5 + g_x(x - x_0)](S_w - 1), \tag{2}$$

$$v = [0.5 + g_y(y - y_0)](S_h - 1). \tag{3}$$

Default gains are 4 and 6. Coordinates are clamped two pixels inside the screen boundary before filtering. This mapping provides controllable movement from image displacement, not a calibration between gaze rays and screen targets.

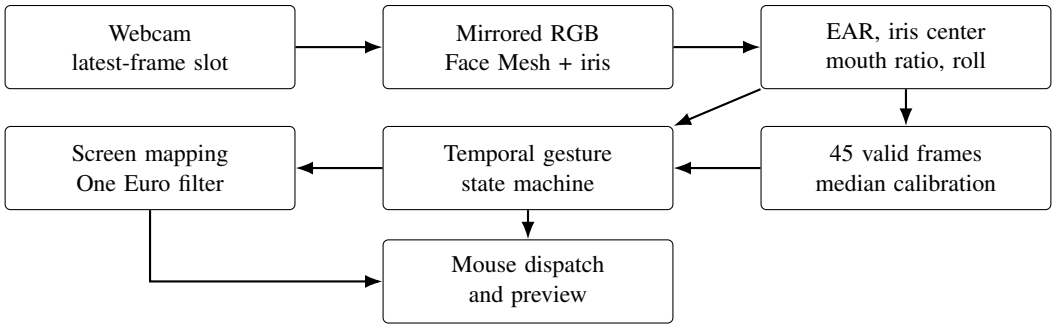



Figure 1. Overview. The gesture policy permits pointer movement only in an appropriate open-eye state. Clicks, scrolling and toggling bypass coordinate filtering. Face loss resets gesture tracking and stops movement.

### 4.3. Temporal Gesture Policy

Each eye closes when $E/E_0 < 0.55$ and reopens when the ratio reaches 0.75. This hysteresis avoids rapid state changes around one threshold. A wink is eligible only after both eyes have been open for 0.12 seconds. Exactly one eye must close for 0.15–1.0 seconds, and a click is emitted on reopening, subject to a 0.35-second cooldown. If both eyes close at any time, the pending wink is cancelled. The pointer holds while the eyes are closed and during rearming.

Mouth ratio above 0.45 starts a hold. After 0.6 seconds the active state toggles once; a latch prevents repeated toggling while the mouth stays open. A ratio at or below 0.28 releases the mouth state. Normal motion remains blocked until the mouth closes. Thus the toggle occurs at the hold threshold, not at mouth release.

Head roll relative to neutral enters scrolling beyond $12^\circ$ and exits at $7^\circ$ or less. After a 0.18-second hold, the controller emits three scroll units at intervals of at least 0.20 seconds. Pointer movement holds during scrolling. Mouth handling takes precedence over eye gestures, and eye closure suppresses scrolling. Face loss or a gap above 0.5 seconds resets gesture state, preventing stale partial gestures from completing later.

### 4.4. Adaptive Smoothing

For each screen coordinate, let $z_t$ be raw input and $\Delta t$ the elapsed time. Compute raw velocity $d_t = (z_t - z_{t-1})/\Delta t$, then low-pass filter it at 1 Hz to obtain $\widehat{d}_t$. The position cutoff and update are

$$f_t = f_{\min} + \beta|\widehat{d}_t|, \tag{4}$$

$$\alpha(f, \Delta t) = \frac{1}{1 + (2\pi f \Delta t)^{-1}}, \tag{5}$$

$$\widehat{z}_t = \widehat{z}_{t-1} + \alpha(f_t, \Delta t)(z_t - \widehat{z}_{t-1}). \tag{6}$$

The defaults are $f_{\min} = 1.2$ Hz and $\beta = 0.007$ for pixel-valued input. Low speed receives stronger smoothing; larger speed raises the cutoff. Filter state resets after tracking interruptions and gesture transitions. This reduces reuse of stale samples but may produce a position jump on re-entry, which requires live testing.

### 4.5. Implementation and Interface

`imouse/core.py` contains geometry and control, `capture.py` owns the reader, and `app.py` handles inference, desktop dispatch, preview, and CSV/JSON logs. The entry point is `mouse-controller.py`. Dependencies pin MediaPipe 0.10.21, OpenCV-contrib 4.11.0.86, NumPy 1.26.4, and PyAutoGUI 0.9.54. Python 3.11/3.12 is the project's documented environment. The preview shows active/paused/dry-run state, gestures, landmarks, processing FPS, and inference time. Desktop mode starts paused. SPACE toggles, C recalibrates, and Q/ESC exits when the preview has focus. The physical-mouse corner fail-safe remains available. A Tkinter target task provides ten targets of radius 38 pixels in a nominal $1000 \times 700$ window.

## 5. Experiments and Results

### 5.1. Evidence and Reproducibility

The numerical timing results below come from the supplied `results/validation_report.json` and corresponding CSV, not a newly measured webcam session. The logged environment is Python 3.12.14 on Linux x86_64, MediaPipe 0.10.21 and OpenCV 4.11.0. CPU model is not recorded, limiting cross-machine comparison. For this documentation, all 33 supplied pytest cases were rerun successfully and the deterministic synthetic experiment was reproduced with the same numerical output. Live webcam/desktop operation was not exercised here.

**Data used.** No task-specific training or fine-tuning was performed. The offline fixture repeats a public NASA astronaut image distributed by scikit-image [7]: 90 face frames, 30 blank frames, then 60 face frames, with a small horizontal translation in part of the first segment. The video is

| Stage | Median (ms) | p95 (ms) |
|---|---|---|
| Capture wait | 1.212 | 2.681 |
| Preprocessing | 0.178 | 0.538 |
| Inference | 4.969 | 10.480 |
| Control | 0.072 | 0.204 |
| Dry-run dispatch | 0.0012 | 0.0020 |
| Read-to-dispatch | 5.328 | 11.573 |
| Loop | 6.661 | 12.930 |

Table 2. Supplied offline fixture timings over all frames, including calibration and no-face frames. These exclude real desktop response.

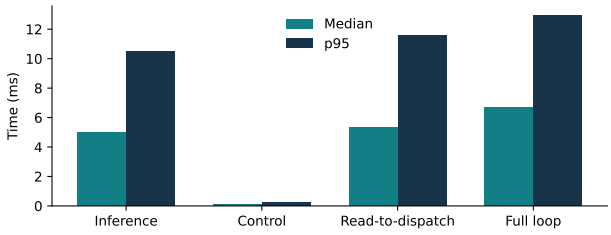



Figure 2. Selected supplied timing distributions summarized by median and p95. Dry-run file replay only.

$640 \times 480$ with a 30 FPS time base. Synthetic feature sequences exercise blinks and winks. Neither fixture is a representative human gesture dataset. The image source and hash are preserved in the supplied result JSON.

### 5.2. Offline Pipeline Timing

All 180 fixture frames were processed, with face features on 150 frames. The recorded throughput is 130.14 processed frames/s for unpaced file replay. It is not a webcam FPS claim. No native mouse actions occurred because the run used dry-run dispatch. Read-to-dispatch begins after the camera/file read completes and includes any wait in the application slot. It excludes exposure, driver buffering and display response. Inference dominates the listed measured compute stages. Per-stage percentiles cannot be added to derive a percentile of total latency. True end-to-end response would require an external measurement of physical action and screen response.

### 5.3. Isolated Library Pause

The supplied microbenchmark executes PyAutoGUI's installed `_handlePause` function for 30 samples at each setting. At 0.1 seconds, the median is 100.128 ms and p95 is 100.205 ms. At zero, these are 0.061 ms and 0.082 ms. This isolates the library sleep; it excludes native mouse calls, perception and display. Removing the pause eliminates an avoidable component but cannot be described as a corresponding speedup of the complete interface. The original

| Scenario | Trials | Emitted clicks |
|---|---|---|
| Bilateral blinks | 100 | 0 |
| Deliberate left winks | 30 | 30 left |
| Deliberate right winks | 30 | 30 right |

Table 3. Reproduced synthetic-feature checks, including asymmetric onset/release of bilateral blinks. Counts establish these scripted behaviors, not human accuracy.

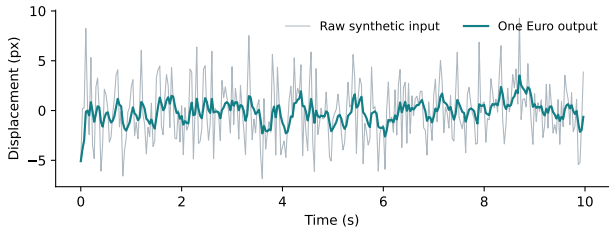



Figure 3. Reproduced stationary noise experiment at 30 Hz. The grey trace is raw input and the teal trace is filtered output. Lower variation does not by itself quantify movement lag.

PDF's roughly 100–217 ms API-call timings use a different scope and are not a matched baseline comparison.

### 5.4. Synthetic Gesture and Jitter Checks

The generator advances time at 30 Hz and includes asymmetric blink transitions. It directly supplies features, bypassing image detection noise. A separate seed-604 Gaussian coordinate trace contains 300 samples, with a nominal 3-pixel noise scale. Excluding the first 30 samples, standard deviation falls from 2.899 to 1.010 pixels, a 65.2% reduction computed as $100(1 - 1.010/2.899)$. The result concerns a stationary synthetic coordinate, not two-dimensional live pointer error or target acquisition speed.

### 5.5. Required Live Follow-up

A matched evaluation should alternate original and revised controllers on the same camera, user, lighting, resolution, and target sequence. Log intended actions independently of emitted events. Per participant, a practical protocol includes 20 winks per eye, 30 ordinary blinks, 10 scroll attempts per direction, 10 toggles, and repeated ten-target tasks. Report participants and repetitions explicitly; these are proposed counts, not completed trials.

Useful outcomes are false clicks during ordinary blinks, gesture precision/recall with independently labeled intentions, hit fraction $H/(H + M)$ for targets, and median/p95 acquisition time from target appearance to its first successful hit. Avoid summing the per-miss timestamps: they are cumulative since the same target appeared. Change one mechanism at a time with `--no-smoothing`,

`--serial-camera`, or `--mouse-pause 0.1`. Measure live stationary jitter and movement lag together. No live trial values are asserted in this report.

## 6. Limitations and Conclusion

The revision makes the control policy explicit and testable. Software checks support blink cancellation, one click per qualifying wink and noise attenuation, while supplied timing logs identify an avoidable library delay. The design still relies on camera-image displacement, user calibration, and visible facial features. Glasses, dim lighting, rapid movement, inability to wink, and fatigue remain unmeasured. Neutral calibration does not compensate for subsequent large changes in pose or distance. Frame skipping can miss a brief gesture, so latency and recognition must be evaluated together. A control-state overlay and target demo exist, but their usability is unvalidated. Future work should prioritize a consented user comparison and alternative click inputs before adding more gestures or claiming accessibility benefit.

## Individual Contributions and Repository

**Research:** All four members independently researched the project and relevant methods.
**Personal contribution (Akshat Singhania):** Independently researched the project and contributed to additional controller features, including calibration, gesture handling, scrolling, pause/resume, and pointer stabilization.
**Iris tracking and pointer control:** Deep Aryan Singh and Harsh Pratap Singh worked on iris-mesh tracking and mouse-movement control.
**Additional functionality:** Akshat Singhania and Rohit Vinod Atkurkar worked on the controller's additional features, including calibration, gesture handling, scrolling, pause/resume, and pointer stabilization.
**Final project repository:** https://github.com/harshpsi9194/EE604_Course-Project.

## References


[1] Google MediaPipe. Face Mesh documentation and source. https://chuoling.github.io/mediapipe/solutions/face_mesh.html; https://github.com/google-ai-edge/mediapipe.

[2] Google MediaPipe. Iris documentation. https://chuoling.github.io/mediapipe/solutions/iris.html.

[3] T. Soukupova and J. Cech. Real-Time Eye Blink Detection using Facial Landmarks. CVWW, 2016. https://cmp.felk.cvut.cz/ftp/articles/cech/Soukupova-CVWW-2016.pdf.

[4] G. Casiez, N. Roussel, and D. Vogel. 1 Euro Filter: A Simple Speed-based Low-pass Filter for Noisy Input in Interactive Systems. CHI, 2012, pp. 2527–2530. https://gery.casiez.net/1euro/; https://github.com/casiez/OneEuroFilter.

[5] A. Sweigart. PyAutoGUI documentation and source. https://pyautogui.readthedocs.io/en/latest/index.html; https://github.com/asweigart/pyautogui.

[6] OpenCV. VideoCapture documentation and source. https://docs.opencv.org/4.x/d8/dfe/classcv_1_1VideoCapture.html; https://github.com/opencv/opencv.

[7] NASA / scikit-image. Astronaut example image. https://scikit-image.org/docs/stable/api/skimage.data.html#skimage.data.astronaut; versioned image: https://github.com/scikit-image/scikit-image/blob/v0.24.0/skimage/data/astronaut.png.

[8] Additional dependency sources. NumPy: https://github.com/numpy/numpy; JAX: https://github.com/jax-ml/jax; pytest: https://github.com/pytest-dev/pytest.

## A. Complete Project Code

The following listings reproduce every Python source file supplied in the project ZIP, including the baseline, application, demo, validation tool and tests. Dependency manifests follow. Third-party dependency source is available through the cited repositories and is not duplicated here. The accompanying source bundle contains raw measurements and documentation. Published revision: https://github.com/harshpsi9194/EE604_Course-Project.

### A.1. mouse-controller.py

```
"""Run: python mouse-controller.py (see README.md for setup and controls)."""
from imouse.app import main


if __name__ == "__main__":
    raise SystemExit(main())
```

### A.2. imouse/__init__.py

```
"""iMouse: webcam face landmarks to desktop mouse actions."""
```

### A.3. imouse/core.py

```
"""Geometry and gesture logic, independent of the webcam and operating system.

All timers use monotonic seconds. No sleeps or mouse calls belong in this file.
"""
from dataclasses import dataclass
import math
from statistics import median


@dataclass(frozen=True)
class Features:
    x: float
    y: float
    left: float
    right: float
    mouth: float
    roll: float


def extract_features(landmarks, width, height):
    """Use pixel distances so ratios do not depend on frame aspect ratio.

    Left/right preserve the original mirrored-input gesture groups. Iris centers
    are 468 and 473; 477 in the original code is an iris boundary point.
    """
    if len(landmarks) < 478 or width <= 0 or height <= 0:
        return None

    def point(index):
        p = landmarks[index]
        return p.x * width, p.y * height

    def distance(a, b):
        return math.dist(point(a), point(b))

    left_width = distance(33, 133)
    right_width = distance(362, 263)
    mouth_width = distance(61, 291)
    if min(left_width, right_width, mouth_width) < 2:
        return None
    left = (distance(160, 144) + distance(158, 153)) / (2 * left_width)
    right = (distance(385, 380) + distance(387, 373)) / (2 * right_width)
    # Eye corner midpoints avoid eyelid motion being mistaken for head tilt.
    a = tuple((u + v) / 2 for u, v in zip(point(33), point(133)))
    b = tuple((u + v) / 2 for u, v in zip(point(362), point(263)))
    if b[0] <= a[0]:
        return None
    roll = math.degrees(math.atan2(b[1] - a[1], b[0] - a[0]))
    iris_a, iris_b = landmarks[468], landmarks[473]
    values = ((iris_a.x + iris_b.x) / 2, (iris_a.y + iris_b.y) / 2,
              left, right, distance(13, 14) / mouth_width, roll)
```

```
    if not all(math.isfinite(v) for v in values):
        return None
    if not (0 <= values[0] <= 1 and 0 <= values[1] <= 1):
        return None
    return Features(*values)


@dataclass(frozen=True)
class Calibration:
    x: float
    y: float
    left: float
    right: float
    roll: float

    @classmethod
    def from_samples(cls, samples):
        if len(samples) < 15:
            raise ValueError("Calibration needs at least 15 valid open-eye frames.")
        values = [median(getattr(s, field) for s in samples)
                  for field in ("x", "y", "left", "right", "roll")]
        if min(values[2:4]) < 0.12:
            raise ValueError("Keep both eyes open during calibration.")
        return cls(*values)


@dataclass(frozen=True)
class Config:
    gain_x: float = 4.0
    gain_y: float = 6.0
    close_ratio: float = 0.55
    reopen_ratio: float = 0.75
    wink_min: float = 0.15
    wink_max: float = 1.0
    click_cooldown: float = 0.35
    rearm_seconds: float = 0.12
    mouth_open: float = 0.45
    mouth_close: float = 0.28
    mouth_hold: float = 0.6
    scroll_enter: float = 12.0
    scroll_exit: float = 7.0
    scroll_hold: float = 0.18
    scroll_interval: float = 0.20
    scroll_steps: int = 3
    min_cutoff: float = 1.2
    beta: float = 0.007
    smoothing: bool = True
    swap_winks: bool = False


class OneEuro:
    """Speed-adaptive low-pass filter (Casiez, Roussel and Vogel, CHI 2012).

    This implementation uses screen pixels and seconds, so beta has pixel units.
    The derivative is computed from successive RAW input samples.
    """
    def __init__(self, min_cutoff=1.2, beta=0.007):
        self.min_cutoff, self.beta = min_cutoff, beta
        self.reset()

    def reset(self):
        self.t = self.raw = self.value = None
        self.derivative = 0.0

    @staticmethod
    def alpha(cutoff, dt):
        return 1.0 / (1.0 + 1.0 / (2.0 * math.pi * cutoff * dt))

    def __call__(self, value, now):
        if self.t is None:
            self.t, self.raw, self.value = now, value, value
            return value
        dt = max(now - self.t, 1e-6)
        velocity = (value - self.raw) / dt
        a = self.alpha(1.0, dt)
        self.derivative += a * (velocity - self.derivative)
        cutoff = self.min_cutoff + self.beta * abs(self.derivative)
        a = self.alpha(cutoff, dt)
        self.value += a * (value - self.value)
```

```
        self.t, self.raw = now, value
        return self.value


@dataclass
class Decision:
    cursor: tuple | None = None
    click: str | None = None
    scroll: int = 0
    toggled: bool = False
    status: str = "Tracking"


class Controller:
    """Blink rejection, wink release detection, and gesture-aware cursor freezing."""
    def __init__(self, calibration, screen_size=(1920, 1080), config=None, active=False):
        self.calibration = calibration
        self.width, self.height = screen_size
        self.config = config or Config()
        self.active = active
        self.filters = [OneEuro(self.config.min_cutoff, self.config.beta) for _ in range(2)]
        self.last_click = self.last_scroll = -math.inf
        self.last_seen = None
        self.last_cursor = None
        self.reset_tracking()

    def reset_tracking(self):
        self.closed = [False, False]
        self.ready = False
        self.open_since = None
        self.wink_eye = None
        self.wink_since = None
        self.mouth_since = None
        self.mouth_latched = False
        self.mouth_is_open = False
        self.scroll_direction = 0
        self.scroll_since = None
        for filt in self.filters:
            filt.reset()

    def toggle(self):
        self.active = not self.active
        self.reset_tracking()

    def step(self, features, now):
        result = Decision(status="Paused" if not self.active else "Tracking")
        if features is None:
            self.reset_tracking()
            self.last_seen = None
            result.status = "No face - movement stopped"
            return result
        # A long stall must not turn an old partial gesture into a new action.
        if self.last_seen is not None and now - self.last_seen > 0.5:
            self.reset_tracking()
        self.last_seen = now
        c, cal = self.config, self.calibration

        threshold = c.mouth_close if self.mouth_is_open else c.mouth_open
        self.mouth_is_open = features.mouth > threshold
        if self.mouth_is_open:
            if self.mouth_since is None:
                self.mouth_since = now
            if not self.mouth_latched and now - self.mouth_since >= c.mouth_hold:
                self.active = not self.active
                self.mouth_latched = True
                result.toggled = True
            self.ready = False
            self.open_since = None
            self.wink_eye = self.wink_since = None
            self.scroll_direction = 0
            result.status = ("Active" if self.active else "Paused") + " - close mouth"
            return result
        self.mouth_since, self.mouth_latched = None, False
        if not self.active:
            return result

        for i, ratio in enumerate((features.left / cal.left, features.right / cal.right)):
            self.closed[i] = ratio < (c.reopen_ratio if self.closed[i] else c.close_ratio)
        both_open = not any(self.closed)
```

```
        both_closed = all(self.closed)

        if not both_open:
            self.open_since = None
            self.scroll_direction = 0
            # Any bilateral closure cancels the WHOLE gesture, even if one eye
            # opens first. This prevents natural blinks becoming clicks.
            if both_closed:
                self.ready = False
                self.wink_eye = self.wink_since = None
            elif self.ready:
                eye = "left" if self.closed[0] else "right"
                if self.wink_eye is None:
                    self.wink_eye, self.wink_since = eye, now
                elif eye != self.wink_eye:
                    self.ready = False
                    self.wink_eye = self.wink_since = None
            result.status = "Eyes closed - cursor held"
            return result

        # Emit a single click after a deliberate unilateral wink is released.
        if self.wink_eye is not None:
            duration = now - self.wink_since
            if c.wink_min <= duration <= c.wink_max and now - self.last_click >= c.click_cooldown:
                result.click = self.wink_eye
                if c.swap_winks:
                    result.click = "right" if result.click == "left" else "left"
                self.last_click = now
            self.wink_eye = self.wink_since = None
            self.ready = False
            self.open_since = now
        if not self.ready:
            if self.open_since is None:
                self.open_since = now
            if now - self.open_since < c.rearm_seconds:
                result.status = "Hold eyes open"
                return result
            self.ready = True
            for filt in self.filters:
                filt.reset()

        tilt = features.roll - cal.roll
        direction = -1 if tilt > c.scroll_enter else (1 if tilt < -c.scroll_enter else 0)
        if self.scroll_direction and abs(tilt) > c.scroll_exit:
            # Keep the current direction within the hysteresis band.
            direction = 1 if tilt < 0 else -1
        if direction:
            if direction != self.scroll_direction:
                self.scroll_direction, self.scroll_since = direction, now
            if now - self.scroll_since >= c.scroll_hold and now - self.last_scroll >= c.scroll_interval:
                result.scroll = direction * c.scroll_steps
                self.last_scroll = now
            result.status = "Scroll up" if direction > 0 else "Scroll down"
            return result
        if self.scroll_direction:
            for filt in self.filters:
                filt.reset()
        self.scroll_direction = 0

        # Preserve the original head-assisted iris-position mapping, with a
        # calibrated neutral center and gain to reach the whole screen.
        x = (0.5 + (features.x - cal.x) * c.gain_x) * (self.width - 1)
        y = (0.5 + (features.y - cal.y) * c.gain_y) * (self.height - 1)
        x = min(max(x, 2), self.width - 3)
        y = min(max(y, 2), self.height - 3)
        if c.smoothing:
            x, y = [f(v, now) for f, v in zip(self.filters, (x, y))]
        self.last_cursor = (round(x), round(y))
        result.cursor = self.last_cursor
        return result
```

## A.4. imouse/capture.py

```
"""Keep only the latest camera frame instead of queuing stale frames."""
import threading
import time


```

```python
class LatestFrameCamera:
    def __init__(self, capture):
        self.capture = capture
        self.condition = threading.Condition()
        self.stop_event = threading.Event()
        self.latest = None
        self.sequence = 0
        self.error = None
        self.thread = threading.Thread(target=self._read, name="imouse-camera", daemon=True)
        self.thread.start()

    def _read(self):
        try:
            while not self.stop_event.is_set():
                ok, frame = self.capture.read()
                if not ok:
                    raise RuntimeError("Camera stopped returning frames.")
                timestamp = time.perf_counter()  # Read-complete time, NOT exposure time.
                with self.condition:
                    self.sequence += 1
                    self.latest = (self.sequence, timestamp, frame)
                    self.condition.notify_all()
        except Exception as exc:
            with self.condition:
                self.error = exc
                self.condition.notify_all()
        finally:
            self.capture.release()

    def read(self, after_sequence, timeout=2.0):
        with self.condition:
            self.condition.wait_for(
                lambda: self.sequence > after_sequence or self.error is not None,
                timeout=timeout,
            )
            if self.error is not None:
                raise RuntimeError(str(self.error)) from self.error
            if self.sequence <= after_sequence:
                raise RuntimeError("Camera timed out. Check permissions or try --camera 1.")
            return self.latest

    def close(self):
        self.stop_event.set()
        self.thread.join(timeout=2.0)
        # Only the reader releases VideoCapture; releasing during a native read
        # from a second thread can crash some camera backends.
```

## A.5. imouse/app.py

```python
"""Local webcam application. --dry-run also accepts a prerecorded video."""
import argparse
from collections import Counter, deque
import csv
from datetime import datetime
import json
from pathlib import Path
import platform
import sys
import time

from .capture import LatestFrameCamera
from .core import Calibration, Config, Controller, extract_features


def parser():
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument("--camera", type=int, default=0)
    p.add_argument("--video", type=Path, help="Replay video; requires --dry-run")
    p.add_argument("--dry-run", action="store_true", help="No operating-system mouse actions")
    p.add_argument("--no-preview", action="store_true", help="Headless evaluation; requires --dry-run")
    p.add_argument("--width", type=int, default=640)
    p.add_argument("--height", type=int, default=480)
    p.add_argument("--fps", type=int, default=30)
    p.add_argument("--max-frames", type=int, default=0, help="0 means unlimited")
    p.add_argument("--seconds", type=float, default=0, help="0 means unlimited")
    p.add_argument("--gain-x", type=float, default=4.0)
    p.add_argument("--gain-y", type=float, default=6.0)
    p.add_argument("--swap-winks", action="store_true")
```

```
    p.add_argument("--no-smoothing", action="store_true")
    p.add_argument("--serial-camera", action="store_true", help="Disable latest-frame thread for comparison")
    p.add_argument("--mouse-pause", type=float, default=0, help="0.1 restores the original per-call delay")
    p.add_argument("--metrics", type=Path, help="CSV output path; also writes a JSON summary")
    return p


class Mouse:
    def __init__(self, dry_run=False, pause=0):
        self.api = None
        self.screen_size = (1920, 1080)
        self.last_position = None
        if not dry_run:
            try:
                import pyautogui
                self.api = pyautogui
                self.api.FAILSAFE = True
                self.api.PAUSE = pause
                self.screen_size = tuple(self.api.size())
            except Exception as exc:
                raise RuntimeError("Desktop mouse access unavailable. Run locally, or use --dry-run.") from exc

    def dispatch(self, decision):
        if self.api is None:
            return 0
        calls = 0
        self.api.failSafeCheck()
        if decision.cursor is not None and decision.cursor != self.last_position:
            self.api.moveTo(*decision.cursor, duration=0)
            self.last_position = decision.cursor
            calls += 1
        if decision.click:
            self.api.click(button=decision.click)
            calls += 1
        if decision.scroll:
            self.api.scroll(decision.scroll)
            calls += 1
        return calls


class Metrics:
    fields = ["frame", "capture_sequence", "gesture_time_s", "mode", "face", "active", "status",
              "click", "scroll", "cursor_x", "cursor_y", "native_calls", "dropped_frames",
              "capture_wait_ms", "preprocess_ms", "inference_ms", "control_ms",
              "dispatch_ms", "read_to_dispatch_ms", "display_ms", "loop_ms"]

    def __init__(self, path, metadata):
        self.path, self.metadata = path, metadata
        path.parent.mkdir(parents=True, exist_ok=True)
        self.stream = path.open("w", newline="", encoding="utf-8")
        self.writer = csv.DictWriter(self.stream, fieldnames=self.fields, lineterminator="\n")
        self.writer.writeheader()
        self.counts = Counter()
        self.started = time.perf_counter()

    def write(self, row):
        self.writer.writerow(row)
        self.counts["frames"] += 1
        self.counts["face_frames"] += int(row["face"])
        self.counts["dropped_frames"] += row["dropped_frames"]
        if row["click"]:
            self.counts[row["click"] + "_click_events"] += 1
        if row["scroll"]:
            self.counts["scroll_events"] += 1

    def close(self):
        self.stream.close()
        elapsed = time.perf_counter() - self.started
        import numpy as np
        with self.path.open(newline="", encoding="utf-8") as stream:
            rows = list(csv.DictReader(stream))
        stats = {}
        for name in self.fields:
            if name.endswith("_ms") and rows:
                values = np.array([float(row[name]) for row in rows])
                stats[name] = {"median": float(np.median(values)), "p95": float(np.percentile(values, 95))}
        action_stats = {}
        action_rows = [r for r in rows if int(r["native_calls"]) > 0]
        for name in ("dispatch_ms", "read_to_dispatch_ms"):
```

```
            if action_rows:
                values = [float(row[name]) for row in action_rows]
                action_stats[name] = {"median": float(np.median(values)), "p95": float(np.percentile(values, 95))}
        summary = {"metadata": self.metadata, "counts": dict(self.counts), "elapsed_seconds": elapsed,
                   "processed_fps": self.counts["frames"] / max(elapsed, 1e-6), "timings_ms": stats,
                   "native_action_frames": len(action_rows), "native_action_timings_ms": action_stats,
                   "measurement_note": "Read-to-dispatch starts AFTER camera read; excludes sensor/driver and OS display latency. Counts are predictions, NOT accuracy. Dry-run dispatch is NOT OS input latency."}
        self.path.with_suffix(".json").write_text(json.dumps(summary, indent=2), encoding="utf-8")
        print(json.dumps(summary, indent=2))


def draw_overlay(cv2, frame, lines, landmarks, active, cursor, screen_size):
    # Preview legibility is independent of the lower-resolution inference input.
    if frame.shape[1] < 640:
        frame = cv2.resize(frame, None, fx=640/frame.shape[1], fy=640/frame.shape[1])
    if landmarks is not None:
        h, w = frame.shape[:2]
        for index in (468, 473, 33, 133, 362, 263, 13, 14):
            p = landmarks[index]
            cv2.circle(frame, (int(p.x * w), int(p.y * h)), 2, (75, 220, 235), -1)
    import numpy as np
    panel = np.zeros((170, frame.shape[1], 3), dtype=np.uint8)
    panel[:] = (28, 24, 22)
    for i, line in enumerate(lines):
        color = (140, 230, 130) if i == 0 and active else (225, 225, 225)
        cv2.putText(panel, line, (12, 24 + i * 26), cv2.FONT_HERSHEY_SIMPLEX, 0.48, color, 1, cv2.LINE_AA)
    if cursor:
        h, w = frame.shape[:2]
        px, py = int(cursor[0] / screen_size[0] * w), int(cursor[1] / screen_size[1] * h)
        cv2.drawMarker(frame, (px, py), (70, 255, 130), cv2.MARKER_CROSS, 18, 2)
    return np.vstack((frame, panel))


def main(argv=None):
    p = parser()
    args = p.parse_args(argv)
    if (args.video or args.no_preview) and not args.dry_run:
        p.error("--video and --no-preview require --dry-run")
    if min(args.width, args.height) < 160 or args.fps <= 0:
        p.error("Use dimensions of at least 160 pixels and positive FPS.")
    if min(args.gain_x, args.gain_y) <= 0 or min(args.mouse_pause, args.seconds, args.max_frames) < 0:
        p.error("Gains must be positive; pause, time limit, and frame limit cannot be negative.")
    try:
        import cv2
        import mediapipe as mp
    except ImportError as exc:
        print(f"Missing dependency: {exc}. Install requirements.txt in Python 3.11/3.12.", file=sys.stderr)
        return 1
    if not hasattr(mp, "solutions"):
        print("Install mediapipe==0.10.21 from requirements.txt for the Face Mesh API.", file=sys.stderr)
        return 1

    capture = reader = mesh = metrics = None
    preview_open = False
    exit_code = 0
    try:
        mouse = Mouse(args.dry_run, args.mouse_pause)
        if args.video:
            if not args.video.is_file():
                raise RuntimeError(f"Video not found: {args.video}")
            capture = cv2.VideoCapture(str(args.video))
        else:
            backend = cv2.CAP_DSHOW if sys.platform == "win32" else cv2.CAP_ANY
            capture = cv2.VideoCapture(args.camera, backend)
            if not capture.isOpened() and backend != cv2.CAP_ANY:
                capture.release()
                capture = cv2.VideoCapture(args.camera)
            capture.set(cv2.CAP_PROP_FRAME_WIDTH, args.width)
            capture.set(cv2.CAP_PROP_FRAME_HEIGHT, args.height)
            capture.set(cv2.CAP_PROP_FPS, args.fps)
            capture.set(cv2.CAP_PROP_BUFFERSIZE, 1)  # Best effort, backend-dependent.
        if not capture.isOpened():
            raise RuntimeError("Cannot open input. Check webcam permissions, close other camera apps, or try --camera 1.")
        source_fps = capture.get(cv2.CAP_PROP_FPS) or args.fps
        if not (0 < source_fps < 1000):
            source_fps = args.fps
        if not args.video and not args.serial_camera:
```

```
            reader = LatestFrameCamera(capture)
        cv2.setNumThreads(1)
        mesh = mp.solutions.face_mesh.FaceMesh(
            static_image_mode=False, max_num_faces=1, refine_landmarks=True,
            min_detection_confidence=0.6, min_tracking_confidence=0.6)
        if not args.no_preview:
            cv2.namedWindow("iMouse", cv2.WINDOW_NORMAL)
            preview_open = True
        config = Config(gain_x=args.gain_x, gain_y=args.gain_y,
                        smoothing=not args.no_smoothing, swap_winks=args.swap_winks)
        controller = None
        samples = []
        calibrating = True
        calibration_start = time.perf_counter()
        path = args.metrics or Path("results") / (datetime.now().strftime("session_%Y%m%d_%H%M%S_%f") + ".csv")
        metadata = {"mode": "dry-run" if args.dry_run else "desktop",
                    "platform": platform.platform(), "python": platform.python_version(),
                    "mediapipe": mp.__version__, "opencv": cv2.__version__,
                    "options": {k: str(v) if isinstance(v, Path) else v for k, v in vars(args).items()},
                    "source_fps": source_fps, "screen_size": mouse.screen_size}
        metrics = Metrics(path, metadata)
        sequence = count = 0
        started = time.perf_counter()
        recent = deque(maxlen=30)
        last_event, event_until = "No gesture yet", 0
        print("iMouse: look straight ahead, both eyes open, mouth closed for calibration.")
        print("SPACE pause/resume | C recalibrate | Q/ESC quit | hold mouth open: pause/resume")
        while True:
            loop_start = time.perf_counter()
            if args.seconds and loop_start - started >= args.seconds:
                break
            previous_sequence = sequence
            if reader:
                sequence, captured_at, frame = reader.read(sequence)
            else:
                ok, frame = capture.read()
                captured_at = time.perf_counter()
                if not ok:
                    if args.video and count:
                        break
                    raise RuntimeError("Input stopped before a frame could be read.")
                sequence += 1
            read_end = time.perf_counter()
            ratio = min(1.0, args.width / frame.shape[1], args.height / frame.shape[0])
            if ratio < 1.0:
                frame = cv2.resize(frame, None, fx=ratio, fy=ratio, interpolation=cv2.INTER_AREA)
            frame = cv2.flip(frame, 1)
            rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            rgb.flags.writeable = False
            preprocess_end = time.perf_counter()
            prediction = mesh.process(rgb)
            inference_end = time.perf_counter()
            landmarks = prediction.multi_face_landmarks[0].landmark if prediction.multi_face_landmarks else None
            features = extract_features(landmarks, frame.shape[1], frame.shape[0]) if landmarks else None
            # Recorded gestures use video time, independent of replay speed.
            gesture_time = sequence / source_fps if args.video else captured_at
            decision = None
            status = "Calibration: look ahead, eyes open, mouth closed"
            if calibrating:
                if features and features.left > 0.12 and features.right > 0.12 and features.mouth < 0.28:
                    samples.append(features)
                else:
                    samples.clear()
                status += f" ({len(samples)}/45)"
                if len(samples) >= 45:
                    cal = Calibration.from_samples(samples)
                    controller = Controller(cal, mouse.screen_size, config, active=args.dry_run)
                    metrics.metadata["calibration"] = vars(cal)
                    calibrating = False
                    print("Calibration complete. Press SPACE to start desktop control.")
                elif not args.video and time.perf_counter() - calibration_start > 20:
                    status = "Calibration waiting: improve lighting, keep eyes open"
            elif controller:
                decision = controller.step(features, gesture_time)
                status = decision.status
                if decision.click or decision.scroll or decision.toggled:
                    last_event = (f"{decision.click} click" if decision.click else
                                  f"scroll {decision.scroll}" if decision.scroll else "control toggled")
                    event_until = time.perf_counter() + 2
```

```
            control_end = time.perf_counter()
            if not controller or not controller.active or features is None or (decision and decision.toggled):
                mouse.last_position = None
            native_calls = mouse.dispatch(decision) if decision else 0
            dispatch_end = time.perf_counter()
            count += 1
            recent.append(dispatch_end)
            fps = (len(recent)-1) / max(recent[-1]-recent[0], 1e-6)
            active = controller.active if controller else False
            key = -1
            if not args.no_preview:
                lines = [f"iMouse | {'DRY RUN' if args.dry_run else 'ACTIVE' if active else 'PAUSED'} | {fps:.1f} FPS",
                         status, f"Inference: {(inference_end-preprocess_end)*1000:.1f} ms | Event: {last_event if time.perf_counter() < event_until else '-'}",
                         "SPACE pause/resume | C recalibrate | Q / ESC quit",
                         "Wink + release: click | Tilt head: scroll",
                         "Hold mouth open 0.6s: pause/resume"]
                display = draw_overlay(cv2, frame, lines, landmarks, active,
                                       controller.last_cursor if controller else None, mouse.screen_size)
                cv2.imshow("iMouse", display)
                key = cv2.waitKey(1) & 0xFF
            display_end = time.perf_counter()
            cursor = decision.cursor if decision and decision.cursor else ("", "")
            metrics.write(dict(frame=count, capture_sequence=sequence,
                gesture_time_s=gesture_time if args.video else captured_at-started, mode=metadata["mode"],
                face=int(features is not None), active=int(active), status=status,
                click=(decision.click or "") if decision else "", scroll=decision.scroll if decision else 0,
                cursor_x=cursor[0], cursor_y=cursor[1], native_calls=native_calls,
                dropped_frames=max(0, sequence-previous_sequence-1),
                capture_wait_ms=(read_end-loop_start)*1000, preprocess_ms=(preprocess_end-read_end)*1000,
                inference_ms=(inference_end-preprocess_end)*1000, control_ms=(control_end-inference_end)*1000,
                dispatch_ms=(dispatch_end-control_end)*1000, read_to_dispatch_ms=(dispatch_end-captured_at)*1000,
                display_ms=(display_end-dispatch_end)*1000, loop_ms=(display_end-loop_start)*1000))
            if key in (ord('q'), 27) or (args.max_frames and count >= args.max_frames):
                break
            if preview_open and cv2.getWindowProperty("iMouse", cv2.WND_PROP_VISIBLE) < 1:
                break
            if key == 32 and controller and not calibrating:
                controller.toggle()
                mouse.last_position = None
            if key == ord('c'):
                calibrating = True
                samples.clear()
                controller = None
                calibration_start = time.perf_counter()
    except KeyboardInterrupt:
        print("Stopped by keyboard.")
    except Exception as exc:
        if type(exc).__name__ == "FailSafeException":
            print("Stopped: cursor reached a screen corner (PyAutoGUI fail-safe).")
        else:
            print(f"iMouse error: {exc}", file=sys.stderr)
            exit_code = 1
    finally:
        if reader:
            reader.close()
        elif capture is not None:
            capture.release()
        if mesh is not None:
            mesh.close()
        if preview_open:
            cv2.destroyAllWindows()
        if metrics:
            metrics.close()
            print(f"Session measurements saved to {metrics.path}")
    return exit_code
```

## A.6. tools/target_practice.py

```
"""Local target-clicking demo; measures task outcomes, not gaze accuracy/latency."""
import csv
from datetime import datetime
import math
from pathlib import Path
import random
import time
import tkinter as tk
```

```

def main():
    root = tk.Tk()
    root.title("iMouse - target practice")
    root.geometry("1000x700")
    root.configure(bg="#101827")
    label = tk.Label(root, text="iMouse target practice - press SPACE to start",
                     bg="#101827", fg="white", font=("Arial", 17))
    label.pack(pady=15)
    canvas = tk.Canvas(root, bg="#101827", highlightthickness=0)
    canvas.pack(fill="both", expand=True)
    instructions = tk.Label(root, text="Click the green target. Hold your mouth open to pause iMouse. ESC closes this
     window.",
                            bg="#101827", fg="#b4c3dc", font=("Arial",11))
    instructions.pack(pady=12)
    rng = random.Random(604)
    state = {"running":False, "hits":0, "misses":0, "target":(0,0), "shown":0}
    rows = []
    radius = 38
    output = Path("results") / (datetime.now().strftime("session_targets_%Y%m%d_%H%M%S_%f") + ".csv")

    def save():
        if rows:
            output.parent.mkdir(parents=True, exist_ok=True)
            with output.open("w",newline="",encoding="utf-8") as stream:
                writer=csv.DictWriter(stream,fieldnames=list(rows[0]),lineterminator="\n")
                writer.writeheader()
                writer.writerows(rows)

    def target():
        canvas.delete("all")
        w,h=max(canvas.winfo_width(),200),max(canvas.winfo_height(),200)
        x,y=rng.randint(60,w-60),rng.randint(60,h-60)
        state.update(target=(x,y),shown=time.perf_counter())
        canvas.create_oval(x-radius,y-radius,x+radius,y+radius,fill="#43d9a3",outline="")
        canvas.create_text(x,y,text=str(state["hits"]+1),font=("Arial",18,"bold"),fill="#101827")
        label.configure(text=f"Targets: {state['hits']}/10 | Misses: {state['misses']}")

    def start(event=None):
        if not state["running"] and state["hits"] == 0:
            state["running"] = True
            target()

    def click(event):
        if not state["running"]:
            return
        x,y=state["target"]
        hit=math.hypot(event.x-x,event.y-y) <= radius
        rows.append({"target":state["hits"]+1,"hit":int(hit),"target_x":x,"target_y":y,
                     "click_x":event.x,"click_y":event.y,
                     "seconds_since_target_shown":time.perf_counter()-state["shown"]})
        if hit:
            state["hits"] += 1
            if state["hits"] == 10:
                state["running"] = False
                save()
                canvas.delete("all")
                label.configure(text=f"Complete: 10 targets | {state['misses']} misses | Results saved")
                canvas.create_text(canvas.winfo_width()//2,canvas.winfo_height()//2,
                    text="Hands-free target practice complete",fill="#43d9a3",font=("Arial",22))
            else:
                target()
        else:
            state["misses"] += 1
            label.configure(text=f"Targets: {state['hits']}/10 | Misses: {state['misses']}")

    def close(event=None):
        save()
        root.destroy()

    canvas.bind("<Button-1>",click)
    root.bind("<space>",start)
    root.bind("<Escape>",close)
    root.protocol("WM_DELETE_WINDOW",close)
    root.mainloop()


if __name__ == "__main__":
```

```
    main()
```

## A.7. tools/validate_pipeline.py

```
"""Reproducible checks using real Face Mesh inference, without an OS mouse.

python tools/validate_pipeline.py --download-sample
The downloaded NASA image is only a smoke-test fixture, not a gesture dataset.
"""
import argparse
import ast
from dataclasses import replace
import hashlib
import importlib.util
import json
from pathlib import Path
import platform
import sys
import tempfile
import time
import urllib.request

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
from imouse.app import main
from imouse.core import Calibration, Controller, Features, OneEuro


def summary(values):
    import numpy as np
    return {"median_ms": float(np.median(values)), "p95_ms": float(np.percentile(values, 95))}


def measure_library_pause():
    """Run the INSTALLED library's real pause function, without importing X11.

    No fake cursor timings: this ONLY measures the library's post-call sleep.
    """
    source = Path(importlib.util.find_spec("pyautogui").origin).read_text(encoding="utf-8")
    tree = ast.parse(source)
    function = next(n for n in tree.body if isinstance(n, ast.FunctionDef) and n.name == "_handlePause")
    module = ast.Module(body=[function], type_ignores=[])
    output = {}
    for pause in (0.1, 0.0):
        namespace = {"time": time, "PAUSE": pause}
        exec(compile(module, "pyautogui._handlePause", "exec"), namespace)
        values = []
        for _ in range(30):
            start = time.perf_counter()
            namespace["_handlePause"](True)
            values.append((time.perf_counter()-start)*1000)
        output[str(pause)] = summary(values)
    return {"scope": "Actual PyAutoGUI _handlePause only; excludes native mouse APIs, camera, inference and display.",
            "samples_per_setting": 30, "pause_seconds": output}


def synthetic_experiments():
    import numpy as np
    rng = np.random.default_rng(604)
    f = Features(0.5,0.5,0.3,0.3,0.04,0)
    c = Controller(Calibration(0.5,0.5,0.3,0.3,0), active=True)
    natural_blinks, wink_counts = 0, {"left":0,"right":0}
    t = 0.0
    # Include one-eye onset and asymmetric release of ordinary bilateral blinks.
    for _ in range(100):
        sequence = [f]*10 + [replace(f,left=0.03)] + [replace(f,left=0.03,right=0.03)]*3 + [replace(f,right=0.03)] + [
 f]*10
        for sample in sequence:
            t += 1/30
            natural_blinks += c.step(sample,t).click is not None
    for eye in ("left", "right"):
        for _ in range(30):
            sequence = [f]*15 + [replace(f,**{eye:0.03})]*8 + [f]*15
            for sample in sequence:
                t += 1/30
                event = c.step(sample,t).click
                if event:
                    wink_counts[event] += 1
```

```
    raw = 500 + rng.normal(0,3,300)
    filt = OneEuro()
    filtered = np.array([filt(v,i/30) for i,v in enumerate(raw)])
    return {"scope": "Synthetic landmark features; software regression evidence, NOT human gesture accuracy.",
            "bilateral_blink_trials":100, "unintended_clicks":natural_blinks,
            "deliberate_winks_per_eye":30, "emitted_clicks":wink_counts,
            "stationary_input_std_px":float(np.std(raw[30:])),
            "filtered_std_px":float(np.std(filtered[30:]))}


def run():
    p=argparse.ArgumentParser(description=__doc__)
    p.add_argument("--image", type=Path)
    p.add_argument("--download-sample", action="store_true")
    p.add_argument("--output", type=Path, default=ROOT / "results" / "validation")
    args=p.parse_args()
    if not args.image and not args.download_sample:
        p.error("Supply --image or --download-sample.")
    import cv2
    import numpy as np
    source_url="https://raw.githubusercontent.com/scikit-image/scikit-image/v0.24.0/skimage/data/astronaut.png"
    args.output.parent.mkdir(parents=True, exist_ok=True)
    with tempfile.TemporaryDirectory(prefix="imouse-check-") as tmp:
        image_path=args.image
        if image_path is None:
            image_path=Path(tmp)/"astronaut.png"
            urllib.request.urlretrieve(source_url, image_path)
        image=cv2.imread(str(image_path))
        if image is None:
            raise ValueError(f"Cannot read {image_path}")
        canvas=np.zeros((480,640,3),dtype=np.uint8)
        canvas[:,80:560]=cv2.resize(image,(480,480))
        video=Path(tmp)/"fixture.avi"
        writer=cv2.VideoWriter(str(video),cv2.VideoWriter_fourcc(*"MJPG"),30,(640,480))
        if not writer.isOpened():
            raise RuntimeError("MJPG video writer unavailable.")
        for i in range(180):
            # 90 face frames, 30 blank frames, 60 face frames after reacquisition.
            frame=np.zeros_like(canvas) if 90 <= i < 120 else canvas.copy()
            if 60 <= i < 90:
                matrix=np.float32([[1,0,3*np.sin(i/5)],[0,1,0]])
                frame=cv2.warpAffine(frame,matrix,(640,480))
            writer.write(frame)
        writer.release()
        csv_path=args.output.with_suffix(".csv")
        code=main(["--video",str(video),"--dry-run","--no-preview","--metrics",str(csv_path)])
        if code:
            raise RuntimeError("Video application failed.")
        pipeline=json.loads(csv_path.with_suffix(".json").read_text())
        assert pipeline["counts"]["frames"] == 180
        assert 140 <= pipeline["counts"]["face_frames"] <= 155
        assert "calibration" in pipeline["metadata"]
        report={"environment":{"python":platform.python_version(),"platform":platform.platform()},
                "fixture":{"description":"Repeated NASA astronaut still with a small synthetic translation and 30
    blank frames; NOT a live webcam or gesture dataset.",
                           "source":source_url if args.download_sample else "User-supplied image",
                           "sha256":hashlib.sha256(image_path.read_bytes()).hexdigest()},
                "pipeline":pipeline, "pause_microbenchmark":measure_library_pause(),
                "synthetic_experiments":synthetic_experiments()}
        report_path=args.output.parent/(args.output.name+"_report.json")
        report_path.write_text(json.dumps(report,indent=2),encoding="utf-8")
        print(json.dumps(report,indent=2))


if __name__ == "__main__":
    run()
```

## A.8. tests/test_core.py

```
from dataclasses import replace
import math
from types import SimpleNamespace

import pytest

from imouse.core import Calibration, Config, Controller, Features, OneEuro, extract_features


```

```
OPEN = Features(0.5, 0.5, 0.3, 0.3, 0.04, 0)
CAL = Calibration(0.5, 0.5, 0.3, 0.3, 0)


def armed(config=None):
    controller = Controller(CAL, config=config, active=True)
    controller.step(OPEN, 0)
    controller.step(OPEN, 0.2)
    return controller


@pytest.mark.parametrize("eye", ["left", "right"])
def test_deliberate_wink_clicks_once_on_release_and_freezes_pointer(eye):
    c = armed()
    shut = replace(OPEN, **{eye: 0.05}, x=0.8)
    for t in (0.3, 0.4, 0.5):
        result = c.step(shut, t)
        assert result.cursor is None and result.click is None
    result = c.step(OPEN, 0.6)
    assert result.click == eye and result.cursor is None
    assert c.step(OPEN, 0.7).click is None
    assert c.step(OPEN, 0.8).cursor is not None


def test_bilateral_blink_with_asymmetric_opening_never_clicks():
    c = armed()
    for t, f in [(0.3, replace(OPEN, left=0.05)),
                 (0.35, replace(OPEN, left=0.05, right=0.05)),
                 (0.55, replace(OPEN, right=0.05)), (0.65, OPEN)]:
        assert c.step(f, t).click is None


@pytest.mark.parametrize("duration", [0.05, 1.2])
def test_too_short_or_too_long_wink_is_ignored(duration):
    c = armed()
    t = 0.3
    while t < 0.3 + duration:
        c.step(replace(OPEN, left=0.05), t)
        t += min(0.05, duration)
    assert c.step(OPEN, 0.3 + duration).click is None


def test_tracking_loss_cancels_pending_wink_and_rearms():
    c = armed()
    c.step(replace(OPEN, left=0.05), 0.3)
    assert c.step(None, 0.5).cursor is None
    assert c.step(OPEN, 0.6).click is None
    assert c.step(OPEN, 0.6).cursor is None
    assert c.step(OPEN, 0.8).cursor is not None


def test_long_processing_stall_cancels_partial_wink():
    c = armed()
    c.step(replace(OPEN, left=0.05), 0.3)
    result = c.step(OPEN, 1.0)
    assert result.click is None and result.cursor is None


def test_hysteresis_does_not_release_eye_at_intermediate_openness():
    c = armed()
    c.step(replace(OPEN, left=0.1), 0.3)
    assert c.step(replace(OPEN, left=0.19), 0.5).click is None
    assert c.step(replace(OPEN, left=0.26), 0.6).click == "left"


def test_cooldown_prevents_rapid_second_click():
    c = armed()
    c.step(replace(OPEN, left=0.05), 0.3)
    assert c.step(OPEN, 0.5).click == "left"
    c.step(OPEN, 0.63)
    c.step(replace(OPEN, right=0.05), 0.64)
    assert c.step(OPEN, 0.80).click is None


def test_swapping_winks():
    c = armed(Config(swap_winks=True))
    c.step(replace(OPEN, left=0.05), 0.3)
    assert c.step(OPEN, 0.5).click == "right"

```

```
def test_mouth_toggle_requires_hold_and_only_fires_once_until_closed():
    c = armed()
    opened = replace(OPEN, mouth=0.7)
    flags = [c.step(opened, t / 10).toggled for t in range(3, 21)]
    assert sum(flags) == 1 and not c.active
    c.step(OPEN, 2.1)
    flags = [c.step(opened, t / 10).toggled for t in range(22, 32)]
    assert sum(flags) == 1 and c.active


def test_brief_mouth_opening_does_not_toggle():
    c = armed()
    c.step(replace(OPEN, mouth=0.7), 0.3)
    assert not c.step(OPEN, 0.5).toggled
    assert c.active


def test_paused_controller_never_dispatches_gestures():
    c = Controller(CAL)
    for t in range(10):
        result = c.step(replace(OPEN, x=0.8, roll=20, left=0.05), t / 10)
        assert result.cursor is None and result.click is None and result.scroll == 0


@pytest.mark.parametrize("tilt,expected", [(15, -3), (-15, 3)])
def test_scroll_hold_repeat_hysteresis_and_cursor_freeze(tilt, expected):
    c = armed()
    assert c.step(replace(OPEN, roll=tilt), 0.3).scroll == 0
    result = c.step(replace(OPEN, roll=tilt), 0.5)
    assert result.scroll == expected and result.cursor is None
    assert c.step(replace(OPEN, roll=tilt), 0.6).scroll == 0
    assert c.step(replace(OPEN, roll=math.copysign(9, tilt)), 0.8).scroll == expected
    assert c.step(OPEN, 0.9).cursor is not None


def test_neutral_roll_calibration_is_subtracted():
    c = Controller(replace(CAL, roll=10), active=True)
    f = replace(OPEN, roll=10)
    c.step(f, 0)
    assert c.step(f, 0.2).scroll == 0


def test_screen_bounds_avoid_failsafe_corners():
    c = armed(Config(smoothing=False))
    for x, y in [(0, 0), (1, 1), (0, 1), (1, 0)]:
        result = c.step(replace(OPEN, x=x, y=y), 0.3)
        assert 2 <= result.cursor[0] <= 1917
        assert 2 <= result.cursor[1] <= 1077


def test_one_euro_reduces_stationary_jitter_and_resets():
    f = OneEuro()
    data = [100 + (-1)**i * 3 for i in range(100)]
    filtered = [f(v, i / 30) for i, v in enumerate(data)]
    assert max(filtered[30:]) - min(filtered[30:]) < 3
    f.reset()
    assert f(42, 10) == 42


def test_speed_adaptive_filter_responds_faster_than_fixed_filter():
    adaptive, fixed = OneEuro(beta=0.007), OneEuro(beta=0)
    adaptive(0, 0)
    fixed(0, 0)
    assert adaptive(500, 1/30) > fixed(500, 1/30)


def test_calibration_is_robust_to_small_number_of_outliers():
    samples = [OPEN] * 43 + [replace(OPEN, x=0.9)] * 2
    assert Calibration.from_samples(samples).x == 0.5
    with pytest.raises(ValueError):
        Calibration.from_samples([OPEN])


def make_landmarks(scale=1, angle=0, width=640, height=480):
    points = {33:(200,180), 133:(240,180), 160:(212,174), 144:(212,186),
              158:(228,174), 153:(228,186), 362:(300,180), 263:(340,180),
              385:(312,174), 380:(312,186), 387:(328,174), 373:(328,186),
              61:(240,250), 291:(300,250), 13:(270,248), 14:(270,252),
```

```
                468:(220,180), 473:(320,180)}
    result = [SimpleNamespace(x=0.5, y=0.5) for _ in range(478)]
    radians = math.radians(angle)
    for index, (x,y) in points.items():
        x, y = (x-270)*scale, (y-200)*scale
        rx = x*math.cos(radians) - y*math.sin(radians) + 270
        ry = x*math.sin(radians) + y*math.cos(radians) + 200
        result[index] = SimpleNamespace(x=rx/width, y=ry/height)
    return result


@pytest.mark.parametrize("scale,angle,width,height", [(0.5,0,640,480), (1.5,0,640,480), (1,20,640,480), (1,0,1280,720)
    ])
def test_geometric_ratios_are_scale_roll_and_aspect_invariant(scale, angle, width, height):
    f = extract_features(make_landmarks(scale, angle, width, height), width, height)
    assert f.left == pytest.approx(0.3)
    assert f.right == pytest.approx(0.3)
    assert f.mouth == pytest.approx(4/60)
    assert f.roll == pytest.approx(angle)


def test_invalid_meshes_do_not_produce_actions():
    assert extract_features([], 640, 480) is None
    points = make_landmarks()
    points[468].x = float("nan")
    assert extract_features(points, 640, 480) is None
```

## A.9. tests/test_app.py

```
import sys
from types import SimpleNamespace
import threading
import time

import pytest

from imouse.app import Mouse, main
from imouse.capture import LatestFrameCamera
from imouse.core import Decision


def test_mouse_has_no_default_delay_and_preserves_fail_safe(monkeypatch):
    calls = []
    api = SimpleNamespace(PAUSE=0.1, FAILSAFE=False, size=lambda: (1920,1080),
        failSafeCheck=lambda: calls.append("check"),
        moveTo=lambda *args, **kwargs: calls.append(("move", args)),
        click=lambda **kwargs: calls.append(("click", kwargs)),
        scroll=lambda value: calls.append(("scroll", value)))
    monkeypatch.setitem(sys.modules, "pyautogui", api)
    mouse = Mouse()
    assert api.PAUSE == 0 and api.FAILSAFE
    mouse.dispatch(Decision(cursor=(20,30)))
    mouse.dispatch(Decision(cursor=(20,30)))
    mouse.dispatch(Decision(click="right"))
    assert len([c for c in calls if isinstance(c, tuple) and c[0] == "move"]) == 1
    assert ("click", {"button":"right"}) in calls
    assert calls.count("check") == 3


def test_dry_run_does_not_import_desktop_api(monkeypatch):
    monkeypatch.setitem(sys.modules, "pyautogui", None)
    Mouse(dry_run=True).dispatch(Decision(cursor=(10,20), click="left", scroll=3))


@pytest.mark.parametrize("argv", [["--video", "test.mp4"], ["--no-preview"], ["--gain-x", "0"], ["--max-frames", "-1"
    ]])
def test_invalid_cli_combinations_exit_before_hardware_access(argv):
    with pytest.raises(SystemExit) as exc:
        main(argv)
    assert exc.value.code == 2


def test_latest_frame_capture_drops_old_frames_and_releases_camera():
    class FakeCamera:
        released = False
        n = 0
        def read(self):
            time.sleep(0.005)
```

```
            self.n += 1
            return True, self.n
        def release(self):
            self.released = True
    camera = FakeCamera()
    reader = LatestFrameCamera(camera)
    first = reader.read(0)
    with reader.condition:
        assert reader.condition.wait_for(lambda: reader.sequence >= first[0] + 4, timeout=1)
    second = reader.read(first[0])
    reader.close()
    assert second[0] >= first[0] + 4
    assert second[2] == second[0]
    assert camera.released and not reader.thread.is_alive()


def test_camera_failure_is_reported_and_released():
    camera = SimpleNamespace(read=lambda: (False,None), release=lambda: None)
    reader = LatestFrameCamera(camera)
    with pytest.raises(RuntimeError, match="stopped returning"):
        reader.read(0)
    reader.close()
```

## A.10. baseline/mouse-controller.original.py

```
import cv2
import mediapipe as mp
import pyautogui
import time
import sys
# Starting the camera
cam = cv2.VideoCapture(0)
face_mesh = mp.solutions.face_mesh.FaceMesh(refine_landmarks=True)
screen_width, screen_height = pyautogui.size()
cv2.namedWindow("Eye Controlled Mouse", cv2.WINDOW_NORMAL)
# Measuring response time
gesture_data = {
    "Cursor":      {"attempts": 0, "success": 0, "response": 0},
    "Left_Click":  {"attempts": 0, "success": 0, "response": 0},
    "Right_Click": {"attempts": 0, "success": 0, "response": 0},
    "Scroll_Up":   {"attempts": 0, "success": 0, "response": 0},
    "Scroll_Down": {"attempts": 0, "success": 0, "response": 0}
}

previous_left_state  = False
previous_right_state = False
last_click_time  = time.time()
last_scroll_time = time.time()
last_stat_print  = time.time()

#values for clicking and exiting

SCROLL_THRESHOLD = 0.05
DEBOUNCE_TIME    = 0.5
MOUTH_OPEN_THRESHOLD = 0.05

#function that updates the stats

def update_stats(gesture, success, start_ts):
    gesture_data[gesture]["attempts"] += 1
    if success:
        gesture_data[gesture]["success"] += 1
        gesture_data[gesture]["response"] += (time.time() - start_ts) * 1000


#printing the stats
def print_stats():
    print("\nGesture Accuracy Report:")
    for key, s in gesture_data.items():
        at, su = s["attempts"], s["success"]
        if at > 0:
            accuracy = su / at * 100
            avereage = (s["response"] / su) if su > 0 else 0
            print(f"  {key:12s}  Acc: {accuracy:5.1f}% ({su}/{at})   Avg resp: {avereage:.2f} ms")
        else:
            print(f"  {key:12s}  No attempts yet")
    print("-" * 50)

```

```
print("Eye Controlled Mouse is active")
print("Open your mouth wide to exit the program")


start_time = time.time()

#Main loop
while True:
    ret, frame = cam.read()
    if not ret:
        break
    frame = cv2.flip(frame, 1)
    rgb   = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    result   = face_mesh.process(rgb)
    h, w, _ = frame.shape

    if result.multi_face_landmarks:
        landmarks = result.multi_face_landmarks[0].landmark

        #checks if the mouth is open
        mouth_open = False
        try:
            if (landmarks[14].y - landmarks[13].y) > MOUTH_OPEN_THRESHOLD:
                mouth_open = True
                print("Mouth open detected - exiting program")
                break
        except:
            pass  #error handling for open mouth detection

        for index in [145, 159, 374, 386]:
            cx, cy = int(landmarks[index].x * w), int(landmarks[index].y * h)
            cv2.circle(frame, (cx, cy), 3, (0, 255, 255), -1)


        # Moving cursor with respect to the position of the user's eye landmarks
        t0 = time.time()
        gaze = landmarks[477]
        screen_x, screen_y = gaze.x * screen_width, gaze.y * screen_height
        pyautogui.moveTo(screen_x, screen_y)
        update_stats("Cursor", True, t0)
        cv2.circle(frame, (int(gaze.x*w), int(gaze.y*h)), 3, (0,255,0), -1)

        # Left click (left eye wink)
        left_closed = (landmarks[145].y - landmarks[159].y) < 0.006
        if left_closed and not previous_left_state and (time.time()-last_click_time)>DEBOUNCE_TIME:
            t0 = time.time()
            pyautogui.click()
            update_stats("Left_Click", True, t0)
            last_click_time = time.time()
        previous_left_state = left_closed

        # Right click (right eye wink)
        right_closed = (landmarks[374].y - landmarks[386].y) < 0.006
        if right_closed and not previous_right_state and (time.time()-last_click_time)>DEBOUNCE_TIME:
            t0 = time.time()
            pyautogui.rightClick()
            update_stats("Right_Click", True, t0)
            last_click_time = time.time()
        previous_right_state = right_closed

        # Scroll up/down
        if (time.time() - last_scroll_time) > DEBOUNCE_TIME:
            tilt = landmarks[374].y - landmarks[145].y
            if abs(tilt) > SCROLL_THRESHOLD:
                t0 = time.time()
                if tilt < 0:
                    pyautogui.scroll(300)
                    update_stats("Scroll_Up", True, t0)
                else:
                    pyautogui.scroll(-300)
                    update_stats("Scroll_Down", True, t0)
                last_scroll_time = time.time()


    cv2.imshow("Eye Controlled Mouse", frame)
    # Add this code after your cv2.imshow() line in the first example
    key = cv2.waitKey(1) & 0xFF
    if key == ord('q') or key == 27:  # q or ESC key to exit
        break
```

```
133
134     # makes sure that the program prints the stats after an interval of 1 min
135     if (time.time() - last_stat_print) > 60:
136         print_stats()
137         last_stat_print = time.time()
138
139     # Alternative exit method - if run for 5 minutes, automatically exit
140     if time.time() - start_time > 300:  # 5 minutes = 300 seconds
141         print("Session time limit reached - exiting")
142         break
143 print("\nFinal Accuracy Report:")
144 print_stats()
145 cam.release()
146 cv2.destroyAllWindows()
```

## A.11. requirements.txt

```
1 # Python 3.11 or 3.12, 64-bit. Preserve the original MediaPipe Solutions API.
2 mediapipe==0.10.21
3 opencv-contrib-python==4.11.0.86
4 numpy==1.26.4
5 PyAutoGUI==0.9.54
6 # Avoid slow resolution against newer JAX versions that require NumPy 2.
7 jax==0.4.35
8 jaxlib==0.4.35
```

## A.12. requirements-dev.txt

```
1 -r requirements.txt
2 pytest==8.3.5
```